# Algorithmic Efficiency & Recursion Toolkit (AERT)

Data Structures – Unit 1 Assignment

Student Name: Surya Rao
Roll No: 2501730243

Course: B.Tech. Cse(Aiml)
Section: A

Subject Code: ETCCDS202

**Table of Contents**


1. Introduction
2. Abstract Data Type (ADT) and Stack ADT
3. Factorial Using Recursion
4. Fibonacci Algorithm Analysis
    - 4.1 Naive Recursive Fibonacci
    - 4.2 Memoized Fibonacci (Dynamic Programming)
5. Tower of Hanoi
6. Recursive Binary Search
7. Algorithm Paradigms
    - 7.1 Divide and Conquer
    - 7.2 Dynamic Programming
    - 7.3 Greedy Algorithm
8. Conclusion

## 1. Introduction

Data structures and algorithms are fundamental concepts in computer science that help organize data and design efficient solutions to problems. Algorithm analysis allows developers to evaluate the performance of algorithms in terms of time and space complexity.

In this assignment, a small Python toolkit called **Algorithmic Efficiency & Recursion Toolkit (AERT)** was implemented. The toolkit demonstrates important concepts such as recursion, algorithm efficiency, and different algorithmic paradigms.

The implemented components include:

- Stack Abstract Data Type
- Recursive factorial function
- Fibonacci algorithm (naive and memoized versions)
- Tower of Hanoi recursive solution
- Recursive binary search

Each algorithm is analyzed based on its **time complexity and space complexity**.

## 2. Abstract Data Type (ADT) and Stack ADT

An **Abstract Data Type (ADT)** is a theoretical model that defines the behavior of a data structure without specifying its implementation details. It focuses on what operations can be performed rather than how they are implemented.

For example, a **Stack ADT** supports operations such as:

- push(x) – adds an element to the top of the stack
- pop() – removes the top element
- peek() – returns the top element without removing it
- isEmpty() – checks whether the stack is empty
- size() – returns the number of elements in the stack

A stack follows the **Last In First Out (LIFO)** principle, meaning the last inserted element is the first one removed.

Stack operations usually take **O(1) constant time** because they operate only on the top element and do not require traversing the entire structure.

In this assignment, the Stack ADT was also used to store recursive traces such as Tower of Hanoi moves or mid indices in binary search.

### 3. Factorial Using Recursion

Factorial is a mathematical function defined as:

**$n! = n \times (n-1)!$**

The base case is:

**$0! = 1$**

Example:

$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$

The recursive approach reduces the problem into smaller subproblems until the base case is reached.

**Time Complexity**

$O(n)$

This is because the function performs exactly **n recursive calls**.

**Space Complexity**

$O(n)$

The recursive calls are stored in the call stack, which grows linearly with n.

**Big-O, Big-Ω, and Big-Θ**

Big-O = $O(n)$
Big-Ω = $\Omega(n)$
Big-Θ = $\Theta(n)$

Since the number of operations grows linearly with n in every case.

## 4. Fibonacci Algorithm Analysis

The Fibonacci sequence is defined as:

$F(n) = F(n-1) + F(n-2)$

with base cases:

$F(0) = 0$
$F(1) = 1$

### 4.1 Naive Recursive Fibonacci

The naive recursive Fibonacci function repeatedly computes the same values many times.

For example:

To compute F(5), the algorithm recomputes F(3), F(2), and other values multiple times.

**Time Complexity**

$O(2^n)$

The number of recursive calls grows exponentially as n increases.

**Space Complexity**

$O(n)$

Due to the recursion stack.

**Why Naive Fibonacci Is Slow**

Naive recursion performs repeated calculations of the same subproblems. For example, F(3) may be computed many times during execution. This leads to unnecessary recursive calls and exponential growth in computation time.

### 4.2 Memoized Fibonacci (Dynamic Programming)

Memoization improves efficiency by storing previously computed results.

When the Fibonacci function calculates a value for the first time, it stores it in memory. If the same value is needed again, the stored value is returned instead of recomputing it.

**Time Complexity**

O(n)

Each Fibonacci number is calculated only once.

**Space Complexity**

O(n)

Memory is used to store previously computed results.

Memoization significantly improves performance compared to naive recursion.

**5. Tower of Hanoi**

Tower of Hanoi is a classic recursive problem involving three rods and multiple disks.

The rules are:

1. Only one disk can be moved at a time
2. A larger disk cannot be placed on a smaller disk
3. Only the top disk can be moved

The recursive solution works by solving smaller subproblems.

Steps:

1. Move n−1 disks from the source rod to the auxiliary rod
2. Move the largest disk to the destination rod
3. Move n−1 disks from the auxiliary rod to the destination rod

**Number of Moves**

The total number of moves required is:

**$2^n - 1$**

For example:

N = 3
Moves = $2^3 - 1 = 7$

**Time Complexity**

$O(2^n)$

The number of moves doubles with each additional disk.

**Space Complexity**

O(n)

The recursion depth equals the number of disks.

## 6. Recursive Binary Search

Binary search is an efficient searching algorithm that works on **sorted arrays**.

The algorithm repeatedly divides the array into two halves and checks the middle element.

If the key is smaller than the middle element, the search continues in the left half.
If the key is larger, the search continues in the right half.

**Recurrence Relation**

$T(n) = T(n/2) + O(1)$

Solving this recurrence gives:

**Time Complexity = O(log n)**

**Cases**

Best Case
O(1) – when the element is found immediately at the middle.

Average Case
O(log n)

Worst Case
O(log n) – when the search continues until only one element remains.

**Space Complexity**

O(log n) because of recursive function calls.

Binary search is an example of the **Divide and Conquer paradigm**.

## 7. Algorithm Paradigms (Short Reflection)

**Divide and Conquer**

Divide and Conquer solves problems by dividing them into smaller parts, solving each part recursively, and combining the results.

Binary search is a classic example because it divides the array into two halves at each step.

Real-life example:
Searching for a word in a dictionary by opening the book in the middle and choosing the correct half.

**Dynamic Programming**

Dynamic Programming solves problems by storing results of smaller subproblems to avoid recomputation.

Memoized Fibonacci is an example of Dynamic Programming because previously computed Fibonacci values are stored and reused.

Real-life example:
Navigation systems that store shortest paths to avoid recalculating routes.

**Greedy Algorithm**

A greedy algorithm chooses the best option at the current step without considering future consequences.

Example: coin change problem where the largest possible coin is chosen first.

Real-life example:
Selecting the highest value items when packing a bag with limited capacity.

## 8. Conclusion

This assignment demonstrated several important concepts related to data structures and algorithms. The Stack ADT was implemented to understand abstract data types and constant-time operations. Recursive algorithms such as factorial, Fibonacci, Tower of Hanoi, and binary search were implemented to explore recursion and algorithm efficiency.

The analysis showed how naive recursion can be inefficient, while techniques such as memoization significantly improve performance. Understanding algorithm paradigms such as Divide and Conquer, Dynamic Programming, and Greedy strategies helps in designing efficient solutions for real-world problems.

Overall, the toolkit successfully demonstrates fundamental algorithmic concepts and their complexity analysis.